

# RAG와 랭체인

## Day 1 오후 — 개념 정리

리뷰 100건을 아는 챗봇은 어떻게 만드는가

- 대상: RAG 챗봇을 만드는 개발자
- 실습 도메인: 무선 이어폰 고객 리뷰 100건
- 핵심: 용어를 먼저 외우지 않고, 필요해진 다음에 이름을 붙인다

## 생존코딩 오준석

## 우리가 만드는 챗봇

무선 이어폰 리뷰 100건을 읽고 답해 주는 챗봇.

사용자: 운동할 때 러닝머신 뛰면서 써도 귀에서 잘 안 빠지나요?

챗봇: 리뷰를 보면 운동용으로는 평가가 좋습니다.  
헬스장에서 3개월 사용한 분은 고개를 숙여도 빠지지 않았다고 하고,  
러닝 10km 동안 한 번도 빠지지 않았다는 후기도 있습니다.  
다만 계단에서 한쪽이 빠져 분실했다는 리뷰도 한 건 있습니다.

이 답변 하나를 만들기 위해 무엇이 필요한지 하나씩 짚어 나간다.

## 그냥 물어보면 어떻게 될까

Claude에게 리뷰를 주지 않고 그대로 물어보면:

질문

->

Claude

->

그럴듯한 창작

Claude는 이 상품을 본 적이 없다. 그런데도 답은 술술 나온다.

모른다고 말하지 않고 지어내는 것 — 이것을 할루시네이션이라 부른다.

## LLM이 모르는 두 가지

| 한계  | 뜻                | 우리 상황     |
|-----|------------------|-----------|
| 최신성 | 학습 시점 이후의 일을 모른다 | 어제 올라온 리뷰 |
| 폐쇄성 | 공개되지 않은 자료를 모른다  | 우리 쇼핑몰 DB |

리뷰 100건은 **폐쇄성** 쪽 문제다. 인터넷 어디에도 없는 우리만의 데이터다.

모델을 바꿔도 해결되지 않는다. 더 똑똑한 모델도 못 본 자료는 모른다.

## 해결책: 답하기 전에 자료를 쥐여준다

방법은 의외로 단순하다. 프롬프트에 리뷰를 같이 붙여 보낸다.



이 구조에 붙은 이름이 RAG (Retrieval-Augmented Generation) 이다.

파인튜닝이 "시험 전에 밤새 외우기"라면, RAG는 "오픈북 시험"이다.  
새로 외우게 하는 대신, 문제를 보고 해당 페이지를 펴서 보여준다.

QUIZ

## 그럼 리뷰 100건을 전부 프롬프트에 넣으면 되지 않나?

검색이니 벡터니 복잡하게 갈 것 없이,  
리뷰 100건을 통째로 프롬프트에 붙여서 물어보면 안 될까?

```
[리뷰 1] ... [리뷰 2] ... [리뷰 3] ... (중략) ... [리뷰 100] ...
```

질문: 러닝머신에서 안 빠지나요?

옆 사람과 30초만 이야기해 보자.

ANSWER

됩니다. 100건이면 그게 더 낫습니다

진짜 문제는 데이터가 커질 때 생긴다.

|      | 리뷰 100건 | 리뷰 10만 건                  |
|------|---------|---------------------------|
| 컨텍스트 | 들어간다    | 안 들어간다                    |
| 비용   | 질문당 소액  | 질문 1회마다 전체 리뷰 값 지불        |
| 정확도  | 좋다      | 오�히려 떨어진다 (관련 없는 리뷰가 노이즈) |

그래서 필요한 기술은 하나로 좁혀진다 — 질문과 관련 있는 5건만 골라내기.  
RAG의 나머지는 전부 이 목적을 위한 부속이다.

## 키워드 검색으로 고르면 될까

LIKE '%러닝머신%' 로 찾아보면:

질문: "러닝머신 뛰면서 써도 안 빠지나요?"

|      |                            |       |
|------|----------------------------|-------|
| r051 | "헬스장 러닝머신에서 시속 12로 뛰어도..." | -> 찾힘 |
| r031 | "웨이트할 때 고개 숙여도 안 빠지고..."   | -> 놓침 |
| r076 | "조깅 정도는 전혀 문제없어요"          | -> 놓침 |

사람은 셋 다 같은 얘기인 줄 안다. 컴퓨터는 글자만 본다.

필요한 것은 글자 일치가 아니라 의미 일치다.



## 의미를 숫자로 바꾼다 — 임베딩

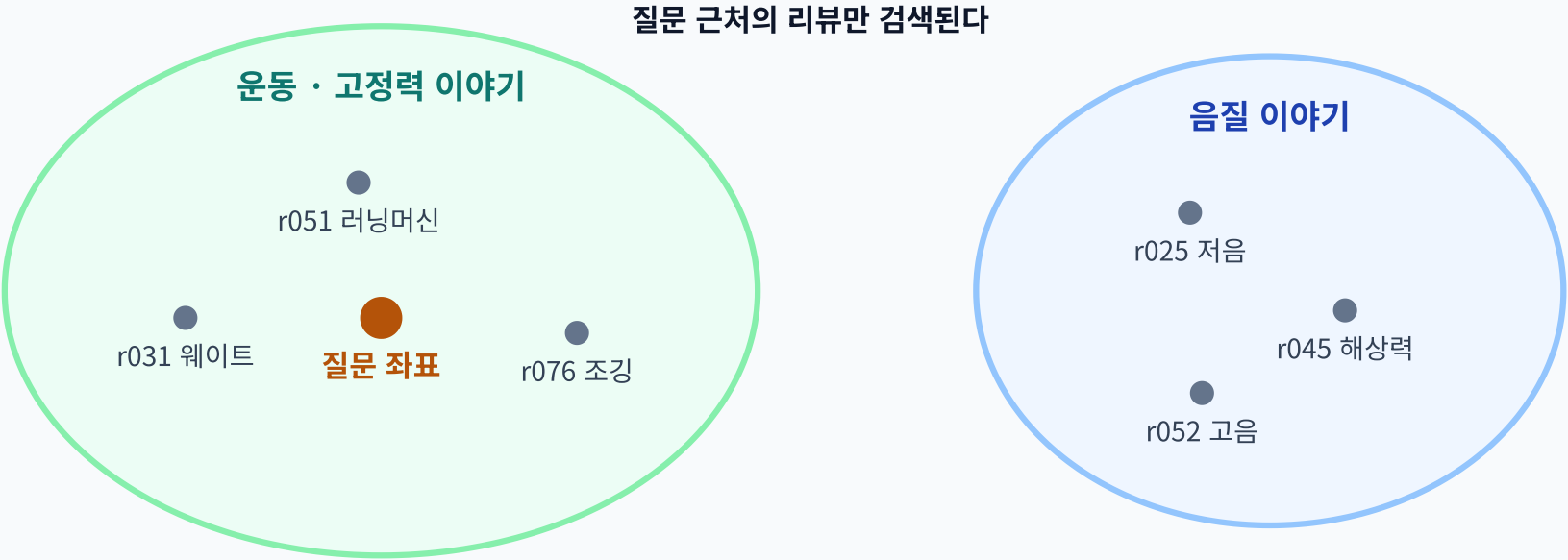
문장을 1024개의 숫자 배열(좌표)로 변환한다.

```
"러닝머신에서 안 빠져요" -> [0.21, -0.05, 0.88, ... ] 숫자 1024개  
"조깅할 때 고정력 좋아요" -> [0.19, -0.07, 0.91, ... ] 비슷한 값  
"저음이 묵직합니다"      -> [-0.62, 0.44, 0.03, ... ] 전혀 다른 값
```

의미가 비슷한 문장은 좌표가 서로 가깝게 찍힌다.

이 변환을 임베딩(Embedding), 만들어진 숫자 배열을 벡터라고 부른다.

# 의미 공간에서 본 리뷰들



질문도 같은 방식으로 좌표를 찍은 뒤, 그 근처의 리뷰 5건을 집어오면 검색 끝.

## 얼마나 가까운가 — cosine

인덱스를 만들 때 고르는 `cosine` 은 두 화살표 사이의 각도를 재는 방식이다.

| 각도         | cosine 값 | 의미         |
|------------|----------|------------|
| 0도 (같은 방향) | 1.0      | 사실상 같은 이야기 |
| 45도        | 0.7      | 관련 있음      |
| 90도 (직각)   | 0.0      | 무관         |

실습 화면의 "89% 일치" 배지가 바로 이 값이다.

길이가 아니라 **방향**을 비교하기 때문에, 문장이 길든 짧은 공정하게 비교된다.

## 벡터 DB는 무엇을 잘하는 DB인가

### PostgreSQL

조건에 정확히 맞는 행 찾기 (WHERE, JOIN, 트랜잭션)

### Pinecone

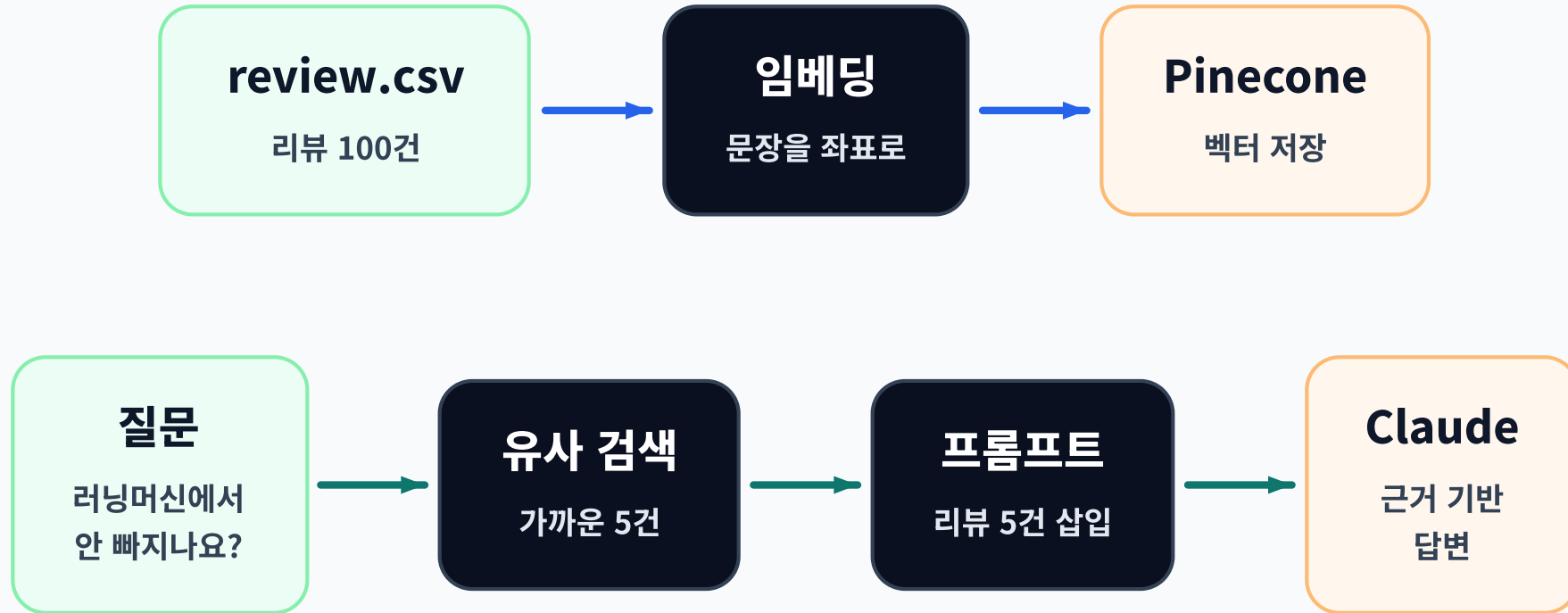
좌표가 가장 가까운 N개 찾기 — 이것 하나를 매우 빠르게

수백만 개 좌표 중 가장 가까운 5개를 밀리초 단위로 찾아준다.  
그래서 우리 실습은 원본 리뷰는 Supabase에, 벡터는 Pinecone에 나눠 담는다.

## 이제 코드로

우리가 만드는 API 두 개가 곧 RAG의 두 단계다.

## RAG의 두 단계



윗줄은 한 번만, 아랫줄은 질문할 때마다 실행된다.

## 준비 단계 — `/api/index-data`

`src/app/api/index-data/route.ts`

1. `review.csv` 100건을 읽는다
2. 각 리뷰의 제목 + 본문을 Pinecone에 넘긴다
3. Pinecone이 좌표로 변환해서 저장한다 `<- upsertRecords()`

화면의 [샘플 데이터 인덱싱] 버튼이 이것을 실행한다.

도서관에 책을 꽂아두는 작업. 딱 한 번만 하면 된다.

## 실행 단계 — `/api/chat`

`src/app/api/chat/route.ts`

1. 질문을 같은 방식으로 좌표 변환      `<- searchRecords()`가 함께 처리
2. 가까운 리뷰 5건 검색      `<- searchRecords()`
3. 그 5건을 프롬프트에 붙인다
4. Claude가 답변 생성      `<- LangChain LCEL`
5. 답변 + 근거 리뷰 5건을 함께 반환      `<- 화면의 "참고 리뷰 카드"`

질문이 들어올 때마다 1~5번이 반복된다.



## 3번에서 실제로 조립되는 것

RAG의 결과물은 결국 문자열 한 덩어리다.

당신은 전자제품 쇼핑물의 전문 리뷰 분석가입니다.  
아래 제공된 [실제 구매 고객 리뷰 데이터]만을 바탕으로 답변하세요.  
리뷰에 없는 사실을 추측하여 지어내지 마세요.

[리뷰 1] 5점 운동할 때 최고 - 러닝하면서 매일 사용하는데 절대 안 빠집니다  
[리뷰 2] 5점 헬스장에서 3개월째 - 고개 숙여도 안 빠지고 땀 범벅이 돼도  
[리뷰 3] 4점 러닝 시 고정력 괜찮아요 - 격한 인터벌만 아니면 잘 버팁니다

질문: 러닝머신 뛰면서 써도 안 빠지나요?

RAG는 마법이 아니다. 검색 결과를 문자열로 붙이는 것이 전부다.

## 우리 실습의 특이점

보통 RAG 강의는 임베딩에 OpenAI를 쓴다. 그런데 우리에게 **Claude** 키만 있다.

**Anthropic은 임베딩 API를 제공하지 않는다. 그래서 그대로는 임베딩을 만들 수 없다.**

### 임베딩 + 검색

Pinecone 통합 임베딩 — 텍스트를 던지면 서버가 좌표까지 만들어 준다 (무료)

### 답변 생성

Claude Haiku 4.5 — 주최측 제공 키

# 랭체인

이제 남은 한 조각. 그런데 이것부터 짚고 가자.

## 랭체인은 AI가 아니다

가장 흔한 오해부터 걷어내자.

### 사람들이 생각하는 것

똑똑한 AI 무언가  
붙이면 성능이 올라가는 것

### 실제

LLM 앱의 반복 잡일을 정리해 둔  
평범한 라이브러리

없어도 된다. `fetch` 로 API를 직접 호출해도 똑같이 동작한다.  
실제로 우리 실습의 검색 부분은 랭체인을 쓰지 않는다.

# 랭체인이 대신해 주는 것

| 부품             | 역할           | 없으면                                 |
|----------------|--------------|-------------------------------------|
| PromptTemplate | 프롬프트 문자열 조립  | 백틱 템플릿으로 직접                         |
| ChatAnthropic  | 모델 호출 어댑터    | <code>fetch</code> 로 직접             |
| OutputParser   | 응답에서 텍스트만 추출 | <code>res.content[0].text</code> 직접 |
| LCEL           | 위 셋을 연결하는 문법 | 함수를 순서대로 호출                         |

넛 다 "못 하던 일"이 아니라 "매번 하던 일"이다.

## LCEL은 유닉스 파이프다

LangChain Expression Language 라는 거창한 이름이 붙었지만, 실체는 파이프다.

```
# 터미널에서 늘 쓰던 것  
cat log.txt | grep ERROR | wc -l
```

```
// LCEL - 똑같은 발상  
prompt.pipe(model).pipe(parser)  
// 질문을 템플릿에 넣고 -> 모델에 보내고 -> 텍스트만 뽑아라
```

이렇게 연결해 두면 스트리밍, 비동기, 중간 단계 로깅이 따라온다.

`.stream()` 한 줄이면 ChatGPT처럼 글자가 타닥타닥 나오게 바꿀 수 있다.

## 진짜 이득은 모델 교체

랭체인을 쓰는 실무적 이유는 사실상 이 한 줄이다.

```
const model = new ChatAnthropic({ model: "claude-haiku-4-5" });  
  
// OpenAI로 갈아타려면 이 줄만 바꾼다. 나머지 코드는 그대로.  
const model = new ChatOpenAI({ model: "gpt-5-nano" });
```

인터넷 예제는 대부분 OpenAI, 우리 수업은 Claude를 쓰는데도  
나머지 코드가 거의 같은 이유가 바로 이것이다.

QUIZ

## 그럼 랭체인은 꼭 써야 하나?

지금까지 본 내용을 정리하면:

- 랭체인은 AI가 아니라 조립 도구다
- 없어도 `fetch` 로 동일하게 만들 수 있다
- 우리 실습도 검색 부분에서는 쓰지 않는다

그런데도 배우는 이유는 무엇일까?



ANSWER

이 규모면 없어도 된다. 다만 —

솔직하게 말하면 이 정도 규모에서는 안 써도 된다. 배워들 값어치는 세 가지다.

| 이유     | 설명                                  |
|--------|-------------------------------------|
| 모델 교체  | 한 줄로 끝난다                            |
| 복잡한 체인 | 대화 기억, 다단계 요약, 도구 호출이 붙으면 직접 짜기 버겁다 |
| 업계 표준  | 남의 AI 코드를 읽으려면 알아야 한다               |

"편해지려고" 쓰는 게 아니라 "바꾸기 쉬우려고" 쓴다.

# 추상화의 함정

편하라고 만든 도구가 방해가 되는 순간.

## 랭체인을 쓰면 안 되는 자리가 있다

랭체인에는 `PineconeStore` 라는 편리한 부품이 있다. 그런데 우리는 못 쓴다.

### PineconeStore의 전제

임베딩은 내 서버에서 만들어 넘긴다 (임베딩 객체 필수)

### 우리 환경

임베딩은 **Pinecone** 서버에서 만든다 (넘길 객체가 없다)

전제가 깨지면 추상화는 도움이 아니라 **장애물**이 된다.

## 그래서 `CLAUDE.md` 에 금지 조항을 넣는다

AI에게 코딩을 맡기면 인터넷에 흔한 방식을 따라간다.

### ## 절대 금지 사항

1. PineconeStore를 사용하지 않는다. (통합 임베딩 인덱스와 비호환)
2. OpenAIEmbeddings 등 외부 임베딩 라이브러리를 추가하지 않는다.
3. 적재는 `upsertRecords()`, 검색은 `searchRecords()`를 사용한다.

내 환경의 전제가 남들과 다르면, 그건 사람이 잡아줘야 한다.

이것이 오후 실습에서 `CLAUDE.md` 를 먼저 쓰는 이유다.

## 오늘의 한 장

### RAG

오픈북 시험 — 질문 -> 관련 자료 5건 검색 -> 프롬프트에 첨부 -> LLM 답변

### 임베딩

의미를 좌표로. 비슷한 뜻이면 가까이 찍힌다

### 벡터 DB

가까운 N개 찾기 하나를 아주 빠르게 (Pinecone)

### 랭체인

배관 도구. `prompt` | `model` | `parser`, 진짜 이득은 모델 교체

### 역할 분담

임베딩·검색 = Pinecone / 답변 생성 = Claude

## 스스로 확인해 보기

스스로 답해 보자. 막히면 앞 슬라이드로 돌아가면 된다.

1. 리뷰 100건을 그냥 프롬프트에 넣으면 안 되는 이유는?
2. 키워드 검색 대신 임베딩을 쓰는 이유를 리뷰 예시로 설명해 보자.
3. `/api/index-data` 와 `/api/chat` 은 각각 몇 번 실행되나?
4. 우리 실습에서 임베딩은 누가 만드나? 왜 그렇게 했나?
5. 랭체인을 빼면 무엇이 불편해지나?

**쇼핑 리뷰 분석 RAG 챗봇 실습에서 이 흐름을 직접 만들어 본다. (약 2시간)**